# System architecture, and the reasoning behind it

**Prepared 20 October 2026.** Companion to *PROJECT_OVERVIEW_AND_PLAN.md*. Describes the system as it stands at commit `9ebaa38`: thirteen Python modules in `hpc_pilots/` (~1,830 lines), a SLURM execution layer, and the document and figure tooling under `output/`.

This document is organised around *why*, not around an API listing. §9 states honestly what runs, what is partial and what is a stub, and §10 lists defects found while writing it.

---

## 1. What the system is for

The system turns public measurement datasets into **graded decision environments** and runs competing decision procedures inside them.

The pattern is the same in every case. Take a dataset in which the outcome is already known for every candidate. Hide all of it behind an evaluator. Let a policy request a few outcomes at a time under a fixed budget, with results optionally delayed by some number of decision rounds. Score what the policy chose against the hidden truth. Because the truth is known in advance, grading is exact, automatic, and requires neither a human reviewer nor a laboratory.

That is the whole idea, and almost all the engineering exists to guarantee one property: **a policy cannot reach an answer it has not earned.** §4 is about how that is enforced.

## 2. The shape, and why it has this shape

Four concerns are kept strictly apart. The separation is not tidiness; each boundary buys something specific.

| LAYER | MODULES | WHAT IT OWNS | WHY IT IS SEPARATE |
|---|---|---|---|
| **Domain** | `dossier.py`, `targets.py`, `chemistry.py` | The seven-axis target dossier, evidence packets for eleven targets across three families, the macrocycle building-block registry | The biology must be representable independently of any policy reasoning over it. `dossier.py` has no I/O and no third-party dependencies at all — it is a schema, and it was written *before* the policy that consumes it. The reverse order produces a schema shaped by whatever the first policy happened to need. |
| **Environment** | `data.py`, `emergency.py` | Dataset staging with provenance; timed decision episodes with evidence release, deadlines and action budgets | This layer holds the hidden truth and exposes only a request interface. It is the auto-gradeable oracle. |
| **Policy** | `binding.py`, `structure.py`, `common.Agent` | The procedures being compared, behind one interface | Comparisons are matched *by construction* rather than by care. All policies share the same surrogate model and the same initial observations at a given seed. |
| **Evaluation** | `calibration.py`, `summarize.py` | Scoring, surrogate audits, aggregation, figures, resource accounting | Kept entirely off the policy path, so a policy cannot read a metric it has not earned. |

A fifth, non-scientific layer under `output/` renders these documents to PDF and draws the figures. It is not part of the research pipeline.

## 3. Module reference

| MODULE | LINES | RESPONSIBILITY |
|---|---|---|
| `common.py` | 171 | Shared infrastructure and the **only** LLM client. Atomic JSON writes, SHA-256 digests, cached downloads with recorded provenance, `.env` loading, and `class Agent`. |
| `data.py` | 289 | Dataset staging. Downloads, validates, encodes and splits every landscape; writes manifests with full provenance. Owns `pool_geometry`, the rule that governs pool construction. |
| `binding.py` | 272 | The acquisition replay engine — the core experiment. Surrogate fitting, the five policies, the delayed-feedback loop, dual scoring, enrichment. |
| `structure.py` | 179 | Structural-evidence allocation over saved MD trajectories. Retired as a scientific component (overview §2.6); the cost-accounting and isolation code is retained. |
| `calibration.py` | 168 | Two surrogate audits, deliberately written plainer than the pilot so they describe the acquisition rule in its simplest form. |
| `dossier.py` | 193 | The decision-state schema: seven axes, four assessment levels, six actions, evidence entries with dates and sources, conflict retention, hard-gate ranking. |
| `targets.py` | 192 | Hand-authored evidence packets: 3 families, 11 targets, 59 evidence entries, 8 synthetic update probes. |
| `emergency.py` | 221 | Timed decision episodes over a packet: evidence released on a schedule, a deadline, an action budget, and scoring of whether the system updated when it should and held when it should not. |
| `chemistry.py` | 135 | Macrocycle building-block registry: initiators, closure chemistry, nine non-canonical block classes, library specification and encoding. |
| `tools.py` | 745 | The tool registry: what a tool is called, how it is reached, what it costs, how it fails, and whether its return may be cited. Deliberately **not** the action space — of the six entries in `dossier.ACTIONS`, three are episode control flow and one is a dossier operation, and none is a callable. `policy_view` projects a spec down to the fields addressed to a decider. Every field is classified into one of three disjoint tables — shown, withheld with a reason, or shown under another name — because conflating the last two made the table claim the policy never learns a tool's cost when it reads one every round. Withheld: module paths, the `trust` annotation that would tell a policy which argument to launder, and the `observed_in` run paths that spell a family as a source literal. `reaches_network` is a capability field separate from the measured `cost_network_calls`, because a count of zero on an unmeasured tool was being read as “touches no network” by a guard — the same split as `cost_seconds` against `worst_case_seconds`. |
| `chief.py` | 418 | Validates a tool call, executes it, and returns an envelope that says what the return is worth. Draws the line between raising and charging exactly where the emitted schema's expressive power ends. |
| `finder.py` | 202 | Which tools a policy may reach this round, and the record of why. Defaults to showing everything, because withholding a tool is a reachability change rather than a cost saving. Its |

| MODULE | LINES | RESPONSIBILITY |
|---|---|---|
| | | stated threshold was corrected from ~65 tools to ~9 once the block was built and measured: one tool's view is 4.9× the whole action block, and 70% of it is the caveats. |
| `optimizer.py` | 644 | Proposes spec corrections from recorded runs, behind a guard on the edits. Propose-only and structurally so: it writes only under `runs/tools/`, which nothing in the package reads. |
| `summarize.py` | 165 | Aggregation into CSVs and figures. Excludes smoke and incomplete runs structurally. |
| `submit.py` | 85 | SLURM submission with a dependency DAG and a submission registry that doubles as an idempotency lock. |
| `stage_model.py` | 29 | Pins a local model by commit SHA with per-file hashes; excludes `*.py` so no model code is downloaded. |
| `__main__.py` | 107 | The CLI. Sixteen subcommands, each lazily importing its implementation so a light command does not pull in scikit-learn or matplotlib. |

## 4. The isolation boundaries — the part that matters most

An evaluation harness whose isolation is a matter of discipline rather than structure will leak, and a leak discovered in week 9 invalidates everything downstream of it. Three boundaries are enforced structurally and each is pinned by a test that would fail if it were breached.

**Outcome isolation.** The full outcome vector never leaves the evaluator. `pick_batch` accepts only revealed values, and top-fraction membership is never supplied to a policy. The test corrupts every *unobserved* label and asserts that the acquisitions are byte-identical — if a policy were reading hidden truth anywhere, that test fails.

**Reference isolation.** For the structural arm, one entire MD replicate (R3) is held out and reachable only inside the evaluator's `measure()`. The test sets R3 to 1000 and asserts an identical decision trace but a different error — the reference moved, the decisions did not.

**Temporal isolation.** In the dossier layer the boundary is a date rather than a value. `Dossier.observe` refuses evidence dated after the decision under consideration, and *records the refusal* rather than discarding it silently. This is what makes the historical COVID-19 replay meaningful rather than a demonstration of hindsight. It is also not sufficient on its own, and the system says so: the state object carries an explicit `hindsight_risk` string noting that naming a real target reintroduces pretrained knowledge no date boundary controls.

**A fourth, weaker boundary: opacity.** Candidate identities in binding prompts are integer pool indices, so a model cannot recognise a variant and recall its phenotype from pretraining. This is a mitigation, not a guarantee, and it is deliberately abandoned in the emergency episodes, where targets must be named for the task to mean anything.

## 5. One episode, end to end

Concretely, `mpro`, UCB policy, pool 0, three-round delay:

**Stage.** `python -m hpc_pilots prepare-target --landscape mpro` reads the landscape spec (`R1AB_SARS2_Flynn_2022`), downloads and caches the ProteinGym metadata and substitutions archive, recording URL, SHA-256, byte count and verification time for each. It reads 5,725 single substitutions, rejects multi-substitution variants explicitly rather than silently coercing them, and **validates every mutation against the reference sequence** — a mismatch raises rather than producing a quietly wrong dataset.

`pool_geometry(5725, 512, 20)` then decides the pool structure. It halves the requested pool size until enough disjoint pools fit, and **shrinks rather than reusing candidates across pools that would then be described as disjoint**. For Mpro it returns 10 disjoint pools of 512 plus a 512-candidate development pool, recording `reduced: true` and `candidates_unused: 93` so the compromise is visible in the manifest rather than buried.

Four files are written: `outcomes.csv`, `features.npy`, `splits.npz`, and a `manifest.json` carrying the measured endpoint, the pool geometry, four SHA-256 hashes, and an explicit `limitations` array.

**Run.** `python -m hpc_pilots target --landscape mpro --pool 0 --policy ucb --delay 3` draws eight seeded initial labels — *identical across all policies at the same seed*, which is what removes initialisation variance from the comparison — then runs fifteen rounds. Each round fits a 64-tree random forest on observed labels only, computes `mean + 1.0σ`, and selects eight candidates. With `delay=3` those land in a pending queue keyed by return round and become visible only when that round arrives.

Tie-breaking permutes the candidate order with a seeded RNG before a *stable* argsort. This sounds like trivia and is not: these landscapes have large exactly-tied plateaus at low budget, and unseeded tie-breaking silently injects a diversity bonus into the greedy policy.

**Score.** At checkpoints of 8/16/32/64/128 labels the evaluator records **both** views: the retrospective metrics over everything *selected*, and the realtime metrics over what has actually *returned*. Keeping both is the single most useful design decision in the system — the gap between them is the project's main surviving finding.

**Stamp and write.** The result is stamped with `dataset_manifest_sha256`, `targets_config_sha256`, `config_sha256` and an `endpoint_warning`, then written atomically.

**Aggregate.** `summarize` collects non-smoke complete runs, grouped by landscape so metrics are never pooled across endpoints, and titles each figure with the endpoint string so the assay cannot be lost from the plot. It also emits `recall_ceiling`, `headroom_multiple` and `attainment` alongside every raw multiple, so a "multiple over random" can never be read without its budget ratio. Those columns exist *because* of the eightfold error.

Two corrections to an earlier version of this paragraph, both of which described the code wrongly. `headroom_multiple` is **not** `1 / labelled fraction` -- that is the formula the overview withdrew in §4.2, and restating it here as current undid the withdrawal. It is `recall_ceiling / labelled_fraction`, where the ceiling prefers the exact counted-hits form `min(1, labels / hits_in_pool)` and falls back to `min(1, labelled / top_fraction)`. And there is **no `normalised_gain` column**; the emitted name is `attainment`, with `policy_attainment` and `random_attainment` beside it. A document naming a column the summariser does not write cannot be checked against the artifact it describes.

## 6. The LLM layer

Exactly one LLM client exists — `common.Agent`. No other module issues a model request.

**Backends.** Any OpenAI-compatible endpoint selected by `PILOT_LLM_PROVIDER`: a local vLLM server, OpenRouter, Anthropic or OpenAI. Per-provider quirks are handled by table rather than by branching — `temperature` is omitted for Anthropic, which rejects it, and the token key becomes `max_completion_tokens` for OpenAI. A hosted run is labelled `agent-<provider>-<model>`, so hosted results can never pool with or overwrite local-model results.

**Constrained decoding.** Actions are forced through a strict JSON schema with the allowed actions as an enum and `additionalProperties: false`. Additional arguments — which target, which axis — are themselves enum-constrained.

**No fallback, ever.** A reply with no JSON object raises. An action outside the allowed set raises. A field outside its enum raises. The class docstring states the rule plainly: *failed inference never substitutes a baseline*, and a test pins it. This matters because the alternative is a silent corruption in which a run labelled "agent" is partly a greedy baseline, and it would be undetectable after the fact. The most recent commit improved the failure message to name the cause, including the specific hint to raise the output ceiling when the reply was truncated.

**Accounting.** Calls, wall seconds, prompt and completion tokens, and the full state/response history are recorded and embedded in every agent run file. LLM latency and tokens are logged separately from compute cost so inference cost and CPU cost never pool.

**What the model actually sees.** Never the hidden outcomes, never the held-out replicate, never top-fraction membership. In an acquisition decision: summary statistics, the sixteen best-predicted candidates by opaque index with prediction, uncertainty heuristic and diversity, and the resource state. In an emergency episode: named targets with per-axis cited evidence — and the hindsight warning.

## 7. Compute and execution

Jobs run on the MATS cluster under SLURM as a dependency-gated chain: a smoke job first, then the binding array (160 tasks) and the ATLAS preprocessing array in parallel, then the structural replays, then the baseline summary, and finally the GPU policy comparison gated on all of it.

Two design points are worth naming:

**The submission registry doubles as a lock.** `runs/submissions.json` is written *after every individual accepted submission*, so a sweep that fails halfway still leaves every accepted job id recorded. Its existence also refuses a second sweep outright — the only sanctioned incremental path is an explicit `--add-agent`. The registry records accepted submissions, not live scheduler state, and the launch record says so: scheduler status must be read from the scheduler.

**Two separate virtual environments.** A 28-package scientific environment and a 157-package vLLM/Torch environment. The scientific environment never has to resolve against CUDA pins, and the smoke suite runs on the small one in under three seconds.

The GPU job mints a 32-byte token into a `0o600` file, binds vLLM to localhost inside the allocation, installs an exit trap to kill the server, polls health while also checking the server process is alive, and runs fully offline with `HF_HUB_OFFLINE` set.

There is **no job-level resume**, and that is a deliberate simplification: each replay writes one self-contained JSON, so re-running a single `(pool, policy, delay)` simply overwrites its file, and `summarize` recomputes completeness from whatever is on disk rather than trusting a progress record.

## 8. Reproducibility machinery, and whether it was worth the cost

Roughly a fifth of the engineering went here. The justification is simple: a result I cannot regenerate is a result I cannot defend at review.

- **Hash everything.** Every download carries URL, SHA-256, bytes and verification time. Every manifest hashes outcomes, features, splits and the config. Every run embeds the dataset manifest hash and config hash.
- **A result names the tool set it ran against.** Every emergency and twins artifact carries a `tool_set_sha256` beside its `config_sha256`, so a spec edit cannot silently reinterpret a finished run. Candidate spec sets go into a new `configs/tools.v{n}.json` rather than editing an existing one, for the same reason `today.json` is treated as byte-immutable, and every earlier snapshot keeps its digest so an old run can still be resolved to the specs it named. Absence of the key means recorded-before-the-registry, not ran-with-an-empty-registry, and it is deliberately not backfilled.
- **Showing a policy the tool specs is its own arm.** The specs reach the decision state only under `PILOT_TOOL_STATE=1`, and that run is labelled `agent-toolstate` so it can never pool with the ~40 recorded episodes. The label reaches the artifact filename, the artifact's top-level `policy`, and — since the defect this arm exposed was fixed — the `policy` column the emergency summary groups by; `policy_requested` keeps the string the caller asked for beside it. The reason is that the schema is unmoved and the prompt is not: the input text changes, so no episode recorded before it is comparable with one after it even at identical payload bytes. The environment states the request and the caller states whether it can honour it, and the effective value is the conjunction: only `emergency` and `twins` build a state that can carry the block, while `binding` and `structure` construct the same `Agent` and never consult it, so reading the variable alone put `-toolstate` on arms that had shown the policy nothing. That is not hypothetical — `slurm/agent.sbatch` exports its environment once and then runs `binding`, `structure`, forty `binding-array` tasks and `summarize`, and never runs either family that honours the flag, so one exported variable would have mislabelled every artifact it produced and then aggregated them. A module builds a tool block if and only if it declares it can, asserted over every module in the package rather than a named few, so a fifth caller fails in either direction. A typo in the variable raises instead of reading as off — off being the arm that pools with the record — while a request an arm cannot honour is recorded rather than raised, because the variable describes an episode's state and `binding` has none. The block shows only tools an action actually dispatches, which is one of the three registered: documentation for work no action can commission is a fabrication channel pointed at the policy. `invariance.certify` scrubs the block alongside the state, and a target name written into a spec's `enum` rather than bound late through `enum_from` fails certification on all four identity elements, which stops the orbit rather than being averaged away. The counterpart report for identifiers nobody declared now covers the same object: it had been computed over the probe records, which are not what a policy reads, so neither the action block nor the tool block was ever scanned for one.
- **Config immutability as a device.** `configs/today.json` is treated as byte-immutable; new work goes into new config files, purely so the `config_sha256` embedded in already recorded runs still verifies.

Both newer configs carry a note saying so.

- **Atomic writes with `allow_nan=False`.** A NaN becomes a write-time error, and a killed job cannot leave a half-written file that the summariser would then parse as a result.
- **Smoke outputs are excluded structurally**, by a flag checked in the aggregator, not by remembering to filter them.
- **Fail loudly, never degrade.** Reference-sequence mismatches raise. Duplicate acquisitions raise. Non-disjoint pools raise. Invalid model actions raise. The summariser reports `complete: false` rather than presenting a partial sweep as a finished one.
- **Epistemic hygiene as a data field, not a convention.** Claims asserted from memory carry `verified=False`; every packet emits a `claims_requiring_citation` list; the dossier's `unsupported()` returns axes asserted without a citation and a test asserts it is empty; and the ranking rationale distinguishes "assessed weak" from "no evidence retrieved", which are very different states. `request_structure` returns an explicit *no structure backend configured* rather than a plausible fabricated structural claim.

**Was it worth it?** For the hashing, atomic writes and no-fallback rules, unambiguously — each has caught a real error. The calibration study is the honest test: it regenerates bit-exactly across all 200 rows of one table and every forest row of the other. One column does not, because the original ridge penalty was not recoverable; the committed script uses `RidgeCV` over a log-spaced grid and reproduces 31 of 160 rows exactly, mean absolute difference 0.0022 on Spearman, no quoted figure affected. That gap is recorded rather than hidden, and it is the argument for having done the rest.

**Tests.** Twenty tests, passing in 2.84 s. They cover the three isolation boundaries, replay invariants (shared initialisation, unique acquisitions, delayed-observation timing), pool disjointness, the shrink-rather-than-overlap rule, the agent HTTP contract and token accounting against a mocked transport, invalid-action failure, calibration label isolation, dossier date refusal and conflict retention, hard-gate ranking, a full emergency episode, and the macrocycle encoding. The suite is deliberately about *invariants that would invalidate a result*, not about line coverage.

## 9. Honest current state

### Working and exercised

- 20 tests passing.
- **Staged data**: GB1 (149,360 rows); three pathogen landscapes — Mpro 5,725 / 10 pools, HA H1 10,715 / 19 pools, HA H3 10,754 / 20 pools; four method-development landscapes with measured binding endpoints, including `spg1_olson_igg` at 536,962 rows, staged specifically to restore the headroom the small pathogen pools cannot provide; the chemistry registry; three family packets; the ATLAS selection manifest.
- **Completed sweeps**: 160 GB1 binding runs across four baselines; 240 pathogen replays (3 landscapes × 4 policies × 10 pools × 2 delays); the calibration studies (200 + 320 rows); nine emergency episodes covering rubric, scripted and a hosted-Claude agent across all three families.

### Partial

- **No non-smoke agent run exists for the binding or structural arms.** The summary status reads 160/200 binding and 3/4 structural — the missing runs are exactly the agent policy. The GPU job was still pending on its dependency at the last checkpoint, and GPU inference has not been validated on an allocation. This is the most important near-term gap.
- **Pathogen sweeps cover the first 10 pools only**, though HA H1 and H3 have 19 and 20 available. Nothing in the config caps this, so it was a budgeted choice rather than a coded one — and it should be recorded as such.
- **Method sweeps are interrupted mid-flight.** Two of four landscapes are complete; one UCB policy has 16 files rather than 20.
- **Summary artifacts are stale** — timestamped before the newest runs, so the aggregate CSVs do not yet include the hosted-agent episodes or the method-landscape gains. Re-running the summariser regenerates all of it.

### Stubs and untouched tracks

- `request_structure` is an honest stub: the request is recorded and charged against the budget, and returns a string saying no structural claim was produced. This is correct behaviour for now — it keeps the resource accounting truthful without fabricating evidence.
- **Every platform-supplied chemistry field is null** — chemical identifier, CCD code, SMILES, stereochemistry, molecular weight, reprogrammed codon, Rosetta parameters, rotamer library — pending the SynX campaign export. The macrocycle encoding is tested but **wired into no replay**. No macrocycle has been represented concretely, let alone measured.

- The **physics-based non-canonical track is untouched**: Flex ddG, rotamer library generation, the RosettaCommons non-canonical interface protocol. Nothing in the repository approaches it.
- Update probes are **synthetic**, and the code says so. They test evidence-following, not the absence of pretrained knowledge.
- The domain layer — targets, dossier, emergency episodes, chemistry, the pathogen and method landscapes — has **no SLURM wiring** and is run by hand. Fine at current scale; it will not survive the episode volumes §1 of the overview implies.

## 10. Defects found while documenting this, and what they turned up

Five were recorded on 20 October; all are closed, and the last turned out to matter far more than it looked.

1. **Dead configuration** — *closed.* `pick_batch` read `forest_trees` and `ucb_beta` from `today.json` unconditionally, so the copies in the pathogen and method configs were never in force. The values coincided, so no recorded result was wrong, but the keys described a control the code did not offer. `replay` and `pick_batch` now take a surrogate argument and each runner passes its own config; the GB1 block remains the default, so recorded replays stay bit-identical. A test switches the exploration bonus off and asserts UCB collapses onto greedy.
2. **Stale `source_manifest.json`** — *closed.* Nine digests had drifted and seven files, including the whole dossier and emergency layer, had never been recorded. It is now regenerable (`python -m hpc_pilots manifest`, with `--check` reporting drift without writing), and `submit.py` prints drift at submission time.
3. **Model manifest path asymmetry** — *closed.* Both paths now resolve through one function, which prefers a staged record and falls back to the committed pin. A second bug surfaced alongside it: staging resolved the revision from the repository head when no staged manifest existed, which in a clean checkout staged whatever was current and defeated the pin.
4. **Unused `initial_labels`** — *closed*, wired in rather than coinciding with the default.
5. **Provenance gap in the run records** — *closed, and it found something.* Every run, episode and audit now carries host, platform, Python version, commit, SLURM identifiers and a `git_dirty` flag. Closing it answered the question it was raised about, and the answer was not the expected one — see below.

### What the provenance gap was hiding

Whether the 160 GB1 results came from the cluster or a local re-run could not be determined from the artifacts. Re-running them locally settled it: **154 of 160 reproduce exactly and six do not — all six in the two forest-backed policies**, with random and diverse reproducing perfectly. Pinned numpy, scipy and scikit-learn all match `requirements.lock.txt`, so the divergence is *below the version pin*: floating-point reduction order differing across machines, flipping a split or a tie.

The authoritative cluster values survive in the SLURM stdout of job 19169 and are recovered into `runs/binding_cluster_finals.csv`, tracked by exception because `runs/` is not. They confirm every GB1 figure the Week 4 debrief quotes. The local run files are now a local re-run and should be re-synced from the cluster.

This matters beyond bookkeeping. A project whose contribution is auditable decision records cannot claim bit-exact reproducibility for its forest-backed policies across machines, and now says so with a number attached.

### Controls added since, each closing a specific way of being wrong

- `summarize_delay` computes the delay effect and writes it with the accounting identity and the feedback-blind null band *beside* it, so the selected-over-returned ratio cannot be read as a measurement again (§4 of the overview).
- The headroom ceiling is corrected to `min(1/top_fraction, 1/labelled_fraction)`, preferring the exact counted-hits form where `hits_in_pool` is recorded, and `attainment` divides by that attainable ceiling rather than by 1.0. (The column is `attainment`; an earlier draft of this list called it `normalised_gain`, which the summariser has never written.)
- A **permuted-outcome control**: outcomes are shuffled before the episode and graded against the shuffle, so a learning policy must fall to random. One that does not is scoring on index order or tie-breaking.
- A **zero-retrieval control** for the dossier arm: retrieval is forbidden and no evidence is pushed, so the policy must decide from an empty dossier against the reference ranking the full evidence base supports. It measures the hindsight risk the code had only ever *named*. The deterministic baselines behave correctly; the LLM named the reference-best target in both families that ran, having bought nothing. Two episodes is not a result, but it is a floor the evidence-using arm must clear before any of its performance can be credited to evidence.
- The action budget could never bind — with deadline 8 and budget 8 at one action per round, the exhaustion branch first held at round 9, which the loop never reached. The budget the prompt told the policy about was fiction.
- `conflict_retained` is a property of the state object, true for every policy including one that ignores the contradiction, and read 1.0 in all nine recorded episodes. It is demoted to a labelled regression check; `addressed_conflict` now asks what the policy did next.

Both controls write to their own files and are excluded from every aggregate, since a falsification check is not a result. Twenty-nine tests pass.

## 11. Running it

```
python -m hpc_pilots prepare                              # GB1 + ATLAS selection manifest
python -m hpc_pilots prepare-target --landscape mpro      # one pathogen landscape
python -m hpc_pilots target --landscape mpro --pool 0 --policy ucb --delay 3
python -m hpc_pilots targets                              # build the evidence packets
python -m hpc_pilots emergency --family coronaviridae --policy agent
python -m hpc_pilots calibration
python -m hpc_pilots summarize
python -m hpc_pilots.submit --with-agent                  # the SLURM chain
```

Model access is configured by environment variable (`PILOT_LLM_PROVIDER`, `PILOT_LLM_MODEL` and a provider key), read from a gitignored `.env`; `.env.example` documents the keys and no secret is in the repository.